MP03 Programación

JavaFX



Centro Oficial FP
Digital & Tech

Ignasi Herrero Espelt

Introducción a JavaFX

> JavaFX es un framework de Java utilizado para crear interfaces gráficas dinámicas e interactivas. Permite desarrollar aplicaciones de escritorio utilizando componentes visuales

- Framework oficial para interfaces gráficas
- Sustituto moderno de Swing
- Compatible con CSS y FXML
- Permite crear aplicaciones multiplataforma
- Basado en programación orientada a objetos

¿Qué es una interfaz gráfica?

- Una interfaz gráfica de usuario (GUI) permite que el usuario interactúe con una aplicación mediante ventanas, botones, menús y otros elementos visuales.

- Antes de JavaFX, Java utilizaba principalmente Swing y AWT para desarrollar interfaces gráficas.

Características principales de JavaFX

- JavaFX incorpora herramientas para crear aplicaciones visuales y fáciles de mantener.

  - Interfaz moderna y personalizable
  - Integración con CSS
  - Uso de archivos FXML
  - Soporte multimedia
  - Animaciones y efectos visuales
  - Arquitectura basada en escenas
  - Compatible con MVC

¿Qué es FXML?

- FXML es un archivo XML que define la estructura visual de la aplicación.
- En FXML se pueden crear:

  - Botones
  - Etiquetas
  - Layouts
  - Campos de texto
  - Tablas
  - Menús

```
<BorderPane xmlns="http://javafx.com/javafx/23.0.1" xmlns:fx="http://javafx.com/fxml/1" fx:controller="
    <top>
        <MenuBar styleClass="menu-bar">
            <menus>
                <Menu text="Archivo">
                    <items>
                        <MenuItem fx:id="newLevelMenuItem" text="Nuevo" />
                        <MenuItem fx:id="exportLevelMenuItem" text="Exporta" />
                        <MenuItem fx:id="exitMenuItem" text="Salir" />
                    </items>
                </Menu>
            </menus>
        </MenuBar>
    </top>
```

Eventos

- JavaFX utiliza un patrón de programación basado en eventos.
- Un evento es una acción que ocurre dentro de la aplicación y que normalmente realiza el usuario.

Ejemplos de eventos:

- Pulsar un botón
- Escribir en un campo de texto
- Mover el ratón
- Seleccionar un elemento
- Cerrar una ventana

```
<Button text="Aceptar"
        onAction="#procesarAccion"/>
```

```
@FXML  new *
private void procesarAccion() {

    System.out.println("Evento ejecutado");
}
```

MVC

- JavaFX utiliza el patrón **MVC (Modelo - Vista - Controlador)** para organizar la aplicación de forma clara y mantenible.

- **Vista (View)**
  Se define mediante archivos **FXML**.
  Contiene la interfaz gráfica: botones, textos, tablas, paneles, etc.

- **Controlador (Controller)**
  Es el encargado de gestionar la interacción entre la vista y el modelo.
  Controla eventos, acciones del usuario y actualiza la interfaz.

- **Modelo (Model)**
  Representa los datos y la lógica de negocio de la aplicación.
  Gestiona la información y el estado de la aplicación.

Conexión entre FXML y el Controlador

- Para que la interfaz funcione correctamente en JavaFX, el archivo **FXML** debe estar vinculado a un controlador.

- Dentro del archivo FXML se indica qué clase actuará como controlador mediante: **fx:controller="paquete.Controlador"**

```
fx:controller="org.example.demo.HelloController">
```

- Cuando el archivo FXML se crea desde herramientas como Scene Builder o IntelliJ, esta referencia suele generarse automáticamente.

Conectar elementos FXML con el controlador

- Para poder manipular los componentes de la interfaz desde Java, utilizamos el atributo **fx:id** en el archivo FXML.
- Cada componente que queramos controlar desde el código debe tener un identificador:

```
<Button fx:id="btnEnviar" text="Enviar"/>
```

- En la clase controlador, el componente se declara utilizando la anotación @FXML

```
@FXML
private Button btnEnviar;
```

Asociar un evento

- Cada componente tiene distintos eventos disponibles (**onAction**, **onMouseClicked**, etc.).
- Para enlazar un evento con el controlador, se utiliza el símbolo **#** seguido del nombre del método:

```
<Button text="Enviar" onAction="#enviarFormulario"/>
```

- En este ejemplo, al pulsar el botón se ejecutará el método **enviarFormulario**.

Asociar un evento

> En el controlador, el método debe declararse con la anotación **@FXML**:

```
@FXML
private void enviarFormulario() {
    System.out.println("Formulario enviado");
}
```

Crear proyecto en JavaFX

- JavaFX al igual que vimos con Junit es un framework por lo que podemos añadirlo directamente desde el Pom.xml

- Crear el proyecto desde 0 puede ser un poco pesado, es por eso que IntelliJ y otros IDEs nos permiten crear una base inicial ya configurada

